uses the password attribute to take the plain-text password, which it encrypts and saves in a database column called encrypted_password.

Devise provides helper methods to simulate a login. Actually, Devise provides two sets of helper methods, both of which use the same method name but are implemented differently. Both kinds are included in the support file shown earlier: Devise::Test::ControllerHelpers for controller tests where you have direct access to the session object, and Devise::Test::IntegrationHelpers for integration, request, and system tests.

You use the sign_in method to sign in a particular user. This user is signed in to the test session and appears as the currently logged-in user for the duration of the test, or until you use the Devise helper method sign_out:

security/01/spec/system/add_project_spec.rb
```
before(:example) do
  sign_in(create(:user))
end
```

Alternatively, you could stub the controller method current_user, which most of the rest of the code uses to determine if there is an active logged-in user.

Creating Users in Fixtures

The user factory depends on factory_bot being able to access attributes that aren't database columns. If you're creating users via fixtures, you don't have that ability, so you need to generate an encrypted password directly. You can do that by setting the encrypted_password attribute directly, as in encrypted_password: <%= Devise::Encryptor.digest(Administrator, 'password!') %>. The code inside the ERB execute markers is there to ensure that the password is being sent to the database encrypted in exactly the way Devise expects so you can match the user on login.

For the integration tests, the sign_in sign-in shortcut has the same name but is implemented differently, using a helper method provided by Warden, which is the tool Devise uses for the authentication functionality. (This method is called login_as, which I mention because you may see it in old code.) Warden works at the Rack layer. Without spending a page or two explaining Rack,[2] suffice to say you can use Warden to fake a logged-in user from outside the Rails stack, suitable for using within your integration tests.

With that addition to the system tests, your suite passes again. (Minitest equivalents are available in the security/01/gatherer/test directory of the code download.) You have nothing to gain right now from trying to move any of

2. https://rack.github.io